

Post-Mortem & Architecture Validation

Gasifier Remaining Useful Life (RUL) & Fix-Before-Fail (FBF) Engine

Prepared by: Dharmadeep

July 2, 2026

Executive Summary

This document details the investigation, resolution, and validation of the machine learning architecture powering the Gasifier RUL Prediction and Fix-Before-Fail (FBF) optimization engine. The investigation resolved a critical data-leakage artifact in late-life inference, validated the Two-Model (Early/Late) architectural strategy, and established empirical proof of the FBF engine's capacity to recognize and reward causal sensor improvements.

1 The Late-Life Inference Anomaly (The Bug)

Initial Observation & Doubt

Doubt: Why did Cross-Validation (CV) show a highly accurate RUL of ~ 20 days at the 100th-day mark, but live inference via the FastAPI endpoint output ~ 1.45 days for the exact same timeframe? Did the model fail to generalize?

Investigation & Conclusion: The model's weights were perfectly sound; the discrepancy was caused by an inference-time data leakage bug known as "Dynamic Scaling." During CV, the feature engineering pipeline normalized the time feature (`age_pct`) by dividing elapsed days by the *known historical total days* (120). Thus, at day 100, the model saw `age_pct` = 83%.

However, in live inference, the pipeline divided elapsed days by the *maximum elapsed days in the incoming batch*. When fed a live batch truncated at day 100, the math evaluated to $100/100 = 100\%$. The model correctly applied its learned rule for a machine at 100% of its life cycle and predicted an immediate shutdown (~ 1.45 days).

Resolution: Replaced dynamic scaling with a static engineering constant based on target plant operations.

```
# The Fix in FastAPI Feature Engineering
CAMPAIGN_LIFE_DAYS = 120.0
g['age_pct'] = g['elapsed_days'] / CAMPAIGN_LIFE_DAYS
```

2 Architecture Validation: The Two-Model Blend

Initial Observation & Doubt

Doubt: If the dynamic scaling bug was present in the live endpoint, why did a live inference test on a 50,000-row batch (Day ~ 35) perfectly predict ~ 79 days RUL without collapsing to zero?

Investigation & Conclusion: This anomaly proved the robustness of the system's **Two-Model Transition Architecture**.

- **Model A (Early Life):** Takes authority before Day 72. During training, Model A was intentionally engineered to be blind to the `age_pct` feature to prevent early-life bias.
- **Model B (Late Life):** Takes authority after Day 72. It relies heavily on `age_pct` to detect minute sensor anomalies near the end of a campaign.

Because the 50,000-row batch occurred during early life, the routing logic passed authority to Model A. Since Model A ignores `age_pct`, it completely bypassed the dynamic scaling bug and accurately predicted the RUL based purely on absolute time and raw sensor physics.

3 Causal Physics Validation (The "Frankenstein" Test)

Initial Observation & Doubt

Doubt: Why use a complex ML model instead of naive math ($120 - \text{elapsed days}$)? If the model relies heavily on `age_pct` in late life, is it simply functioning as an expensive calendar, or does it actually read the physical degradation of the sensors?

Experiment (Data Mutation): To test if the model respects physical sensor health, we executed a data mutation test on a live dataset (g5r19):

1. **Baseline:** Evaluated late-life data ending at Day ~ 100 . (Result: **14.79 days RUL**).
2. **Mutation:** Overwrote the final 1,000 rows of sensor telemetry (temperatures, vibrations, etc.) with pristine, healthy data from the start of the run, while keeping the late-life timestamps intact.

Conclusion: The model recognized the sudden physical improvement in the sensors and increased the RUL prediction to **18.10 days**. This mathematically proves that the FBF engine actively fuses time-series progression with actual equipment health, rather than blindly following a calendar schedule.

4 Prescriptive Maintenance & The FBF Engine

Initial Observation & Doubt

Doubt: In the mutation test, we injected *perfect* brand-new sensor data at Day 100. Why did the RUL only increase to 18.1 days instead of jumping back to 60 or 70 days? If the FBF engine recommends optimal fixes, will the predicted improvements be artificially capped?

Investigation & Conclusion: This behavior is not a flaw; it is the mathematical reality of Tree-Based ML models (like XGBoost) dealing with **Out-of-Distribution (OOD)** data. Tree models cannot extrapolate beyond the boundaries of their historical training data.

When the model receives perfect sensor data at Day 100, it searches its internal logic (historical leaf nodes) for the **"Golden Zone"**—the absolute best outcome *historically recorded* for a gasifier at 100 days of age. If the absolute best historical gasifier only survived another 18.1 days from that point, the model mathematically caps the reward at 18.1 days.

The Operational Reality (The Flywheel Effect): The FBF engine functions as a perfect **Interpolation Engine**. It safely guides operators from a degraded state (e.g., 14 days RUL) back to the safest, most optimal historical baseline (18.1 days). Once operators apply these fixes and physically push the gasifier to run longer than historically precedent (e.g., 135 days total), that new data feeds back into the next training cycle, organically raising the "Golden Zone" ceiling for future predictions.